# 🔵 PHASE 1 — PART 2

# ARRAY CREATION (Deep Mastery Notes)

---

## 1️⃣ Why Array Creation Matters

Array creation is not just about generating numbers.

It determines:

- Memory allocation strategy
- Data type precision
- Performance
- Future computation speed
- Memory efficiency

Poor creation choices → slow models, wasted RAM, precision errors.

## 2️⃣ `np.array()` — Converting Existing Data

### Definition

Creates an ndarray from Python sequences (list, tuple, nested list).

Example:

```
np.array([1, 2, 3])
np.array([[1,2],[3,4]])
```

---

## Important Behaviors

### 2.1 Automatic Type Inference

NumPy determines dtype automatically.

Example:

```
np.array([1, 2, 3.5])
```

Becomes float64 (type promotion).

Rule followed:

Type promotion hierarchy (safe upcasting)

---

# 2.2 Forcing dtype

You can override inference:

```
np.array([1,2,3], dtype=np.float32)
```

Why this matters:

- Controls memory
- Controls precision
- Important in ML & large datasets

---

# 2.3 Copy Behavior

By default:

- It may copy data.
- Use `copy=False` (when safe).

Memory awareness begins here.



**`np.zeros()`**

### Definition

Creates array filled with zeros.

```
np.zeros((3,4))
```

Allocates memory and initializes every element to 0.

---

### Important Insight

Zero initialization takes time.

It writes to every memory location.

## 4 `np.ones()`

Creates array filled with ones.

```
np.ones((2,3))
```

Same memory behavior as zeros — fully initialized.

---

## 5 `np.full()`

Creates array filled with a specific value.

```
np.full((2,2), 7)
```

More flexible than zeros/ones.

Useful when initializing masks, constants, labels.

---

# 6 `np.empty()` (Advanced — Often Ignored)

### Definition

Allocates memory WITHOUT initializing values.

```
np.empty((3,3))
```

Important:

- It does NOT set values to zero.
- It contains random garbage memory.

---

# Why It's Fast

Because:

- It only reserves memory.
- It does NOT write values.

No initialization cost.

---

## When To Use

- When you plan to immediately overwrite all values.
- High-performance scientific computing.

Never use if you rely on default values.

---

## 7 `np.arange()`

### Definition

Creates evenly spaced values within interval.

```
np.arange(start, stop, step)
```

Example:

```
np.arange(0, 10, 2)
```

---

## Important Detail

Stop value is excluded.

---

## Float Warning

Using float steps:

```
np.arange(0, 1, 0.1)
```

May produce precision issues due to floating-point representation.

Because it uses incremental addition.

---

## 8 `np.linspace()`

### Definition

Creates evenly spaced numbers over interval.

```
np.linspace(start, stop, num)
```

Example:

```
np.linspace(0, 1, 5)
```

---

# Key Difference from arange

- arange → uses step
- linspace → uses number of points

---

# Why linspace Is Safer for Floats

It calculates values using formula:

```
(start + step*i)
```

Instead of cumulative addition.

Less floating-point drift.

---



**`np.eye()`**

### Definition

Creates identity matrix.

```
np.eye(3)
```

Produces:
1s on diagonal
0 elsewhere.

---

# Why Important

Used in:

- Linear algebra
- Matrix inversion

- Neural networks
- Covariance matrices

---

# 🔟 `np.random` (Modern Usage)

NumPy now recommends:

```
rng = np.random.default_rng()
```

Instead of older:

```
np.random.rand()
```

---

## Common Random Methods

- rng.random()
- rng.integers()
- rng.normal()
- rng.uniform()

---

## Why Random Generator Object?

Better:

- Reproducibility
- Thread safety
- Performance
- Modern design

---

# 1️⃣1️⃣ dtype Control During Creation

You can specify dtype in almost all creation functions:

```
np.zeros((3,3), dtype=np.float32)
np.arange(10, dtype=np.int16)
np.linspace(0, 1, 5, dtype=np.float64)
```

---

## Why This Is Critical

- float64 = 8 bytes
- float32 = 4 bytes
- int8 = 1 byte

In big arrays:
Memory difference = massive.

Example:
1 million float64 = 8 MB
1 million float32 = 4 MB

---

# 1️⃣2️⃣ Performance Comparison Insight

Fastest to slowest (roughly):

1. np.empty() → fastest (no initialization)
2. np.zeros() → medium
3. np.ones() → medium
4. np.full() → slightly slower
5. np.array() → depends (needs conversion)
6. Python list → slowest

Reason:
Initialization cost + Python object overhead.

---

# 1️⃣3️⃣ Memory Allocation Logic

When you create array:

1. NumPy allocates contiguous memory block
2. Determines dtype
3. Calculates required bytes
4. Initializes (unless empty)

Memory formula:

```
Total bytes = number of elements × itemsize
```

---

# 1️⃣4️⃣ Choosing the Right Function (Professional Thinking)

Use:

- `np.array()` → converting existing data
- `np.zeros()` → initialize with neutral element
- `np.ones()` → scaling or masks
- `np.full()` → constant initialization
- `np.empty()` → performance-critical overwrite
- `np.arange()` → integer sequences
- `np.linspace()` → precise float spacing
- `np.eye()` → identity matrices
- `rng` methods → reproducible randomness

---

# 📌 Professional-Level Warnings

- Never rely on default dtype blindly.
- Avoid float step in arange.
- Avoid np.empty unless overwriting immediately.
- Always think memory in large datasets.
- Prefer default_rng over old random API.

---

# 🔥 Final Mental Model

Array creation is:

Memory allocation + dtype decision + initialization strategy.

If you control these three,
you control performance.